

Current Measurement Characterization

Kenneth TANFA

Abstract

In version 3.4 of the micro-MMC, measurements of the bus voltage (V_{dc}), bridge-leg voltages ($V_{bridge1}$ and $V_{bridge2}$), temperature-sensor voltage, and currents (V_{cm1} and V_{cm2}) must be performed with high precision to mitigate errors and improve control performance. Precise timing is especially critical for V_{cm1} and V_{cm2} , which should be sampled during the motor drive's low-side FET conduction to capture the average current. This document explains how we achieve this and presents the validation test results.

Timing Configuration

Motor Drive Delay Parameters

Based on previous analysis, we estimated the delay introduced by the motor drive to be approximately **535 ns**. This a critical parameter must be compensated in this configuration of various timing requirements.

To accurately determine the execution timing of different code segments, we will perform a dedicated test.

Timer Configuration

Our objective is to trigger the ADC during the low-side FET conduction in order to capture the average current. Using the IOC file, we set up the ADC and timer according to the following configuration:

- System clock frequency : 64 MHz
- Counting Mode: Center Aligned Mode 2
- Prescaler : / 2
- Timer period (ARR): 640 for TIM1,

Based on this configuration, our code should achieve:

TIM1_CCR1 = 320 (PWM OUT1 at 50% duty cycle: $640/2 = 320$)

TIM1_CCR4 = 355 (ADC trigger with delay compensation: $355 - 35 =$) but the test show that it work best with TIM1_CCR4 = 500 . In fact, with this configuration the measurements are close to the expected value

Note: Further testing is needed to precisely determine the correct timing sequence.

Test Methodology

Code Configuration for Testing

The test setup uses a specific configuration to characterize current measurement timing:

```
/* Private typedef -----*/
/* USER CODE BEGIN PTD */
#define TIMER_CLK      64000000
#define TIMER1_ARR      640
#define TIMER2_ARR      1000
#define ADC_RESOLUTION  4096
#define DMA_SAMPLES_BIT 1 // in normal condition it is 3
#define DMA_SAMPLES      1 // In normal condition it is 8
                        //2^DMA_SAMPLES_BIT or (1<<(DMA_SAMPLES_BIT-1))
#define ADC_CHANNELS     6 // Number of adc channels to sample
#define DMA_BUFFER_SIZE  ADC_CHANNELS*DMA_SAMPLES
#define DAC_OFFSET       2048 // 1.65V/3.3*ADC_RESOLUTION
```

We will define the DMA buffer as follows:

```
uint16_t dma_buffer[2 * DMA_BUFFER_SIZE]; // Double buffer for ping-pong operation
uint16_t bufferStart = 0;
uint8_t processData_flag = 0;
```

So let's explain this code

With the following instruction: `#define DMA_BUFFER_SIZE (ADC_CHANNELS * DMASAMPLES)`, we configure **six ADC channel** and **one DMA samples** (`ADC_CHANNELS = 6`, `DMASAMPLES = 6`), resulting in a `DMA_BUFFER_SIZE` of **6**. Since the buffer is defined as `2 * DMA_BUFFER_SIZE`, the total length is **12 samples**.

The following callback functions allow us to efficiently use the double length of the buffer to trigger a test point.

```
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc) {
    TP_SET();
    bufferStart = 0;
    processData_flag = 1;
    TP_RESET();
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
    TP_SET();
    bufferStart = DMA_BUFFER_SIZE;
    processData_flag = 1;
    TP_RESET();
}
```

- **HAL_ADC_ConvHalfCpltCallback** is invoked automatically when **half of the DMA buffer** has been filled with ADC data.
- **HAL_ADC_ConvCpltCallback** is invoked when the **entire DMA buffer** is full. This approach enables us to precisely determine the correct trigger timing.

In this configuration, the first callback occurs when the first 6 samples have been loaded, and the second callback occurs when the remaining 6 samples have been loaded. This structure enables precise measurement of the trigger timing for each buffer fill event.

Test Configuration

For the test, PWM on **OUT1** was configured to a **50% duty cycle**. The **ADC conversion trigger** is scheduled at the desired phase point **with compensation for the ~535 ns motor-drive delay**.

With the selected timer base and prescaler, this yielded the following compare values:

- **TIM1_CCR1 = 320** (PWM timing)
- **TIM1_CCR4 = 500**

Results

Here some results



